

- Dos tipos no pueden tener el mismo nombre, tampoco dos variantes, aunque pertenezcan a tipos diferentes, pueden tener el mismo nombre.
- Para cada variante, un procedimiento constructor es creado. Si hay n campos en una variante, su constructor recibe n argumentos, prueba sí cada uno de ellos satisface el predicado asociado y retorna un nuevo valor de dicha variante del tipo de dato.
- El nombre *nombre-predicado-tipo* es ligado a un predicado. Este predicado determina sí su argumento es un valor perteneciente al tipo nombre-tipo.



Expresiones cálculo λ

```
(define-datatype lc-exp lc-exp?
  (var-exp (id identifier?))
  (lambda-exp (id identifier?)
               (body lc-exp?))
  (app-exp (rator lc-exp?)
            (rand lc-exp?)))
```



Tipo de dato s-list

```
(define-datatype s-list s-list?
  (empty-s-list)
  (non-empty-s-list (first s-exp?)
                    (rest s-list?)))

(define-datatype s-exp s-exp?
  (symbol-s-exp (sym symbol?))
  (s-list-s-exp (slst s-list?)))
```



Tipo de dato s-list

```
(define-datatype s-list s-list?
  (an-s-list (sexps (list-of s-exp?))))

(define list-of
  (lambda (pred)
    (lambda (val)
      (or (null? val)
          (and (pair? val)
                (pred (car val))
                ((list-of pred) (cdr val))))))))
```



- Para determinar a que objeto de un tipo de dato pertenece una variante y extraer sus componentes, se usa la forma *cases*, la cual tiene la sintaxis general:

(cases nombre-tipo expresion

{(nombre-variante ({nombre-campo}) consecuente) }**

(else por-defecto))



- La expresión *cases* evalúa *expresion*. Esto da como resultado un valor *v* de tipo *nombre-tipo*.
- Si *v* es una variante *nombre-variante*, cada uno de los campos *nombre-campo* son asociados al valor del correspondiente campo de *v*. Luego la expresión *consecuente* es evaluada y su valor es retornado.
- Si *v* no es una de las variantes y la cláusula *else* es especificada, la expresión *por-defecto* es evaluada y su valor retornado.
- Si no existe una cláusula *else*, entonces tiene que existir una variante para todos los tipos de dato.



```
(define occurs-free?
  (lambda (search-var exp)
    (cases lc-exp exp
      (var-exp (var)
        (eqv? var search-var))
      (lambda-exp (bound-var body)
        (and (not (eqv? search-var bound-var))
              (occurs-free? search-var body)))
      (app-exp (rator rand)
        (or (occurs-free? search-var rator)
            (occurs-free? search-var rand))))))
```



Tipo de dato bin-tree

```
(define-datatype bintree bintree?  
  (leaf-node (datum number?))  
  (interior-node (key symbol?)  
                 (left bintree?)  
                 (right bintree?)))
```



Tipo de dato bin-tree

Definir un procedimiento que permita encontrar la suma de los enteros en las hojas de un árbol. Usando cases se tiene:

```
(define leaf-sum
  (lambda (tree)
    (cases bintree tree
      (leaf-node (datum) datum)
      (interior-node (key left right)
        (+ (leaf-sum left) (leaf-sum right))))))
```



Ambientes

```
(define-datatype environment environment?
  (empty-env-record)
  (extended-env-record (syms (list-of symbol?))
                        (vals (list-of scheme-value?))
                        (env environment?)))

(define scheme-value? (lambda (v) #t))
```



Ambientes

```
(define apply-env
  (lambda (env sym)
    (cases environment env
      (empty-env-record ()
        (eopl:error 'apply-env
          "No binding for ~s" sym))
      (extended-env-record (syms vals env)
        (let ((pos (list-find-position sym syms)))
          (if (number? pos)
              (list-ref vals pos)
              (apply-env env sym)))))))
```



- Dada la gramática de las expresiones del cálculo lambda:

$$\begin{aligned}\langle \text{expresión} \rangle &::= \langle \text{identificador} \rangle \\ &::= (\text{lambda } (\langle \text{identificador} \rangle) \langle \text{expresión} \rangle) \\ &::= (\langle \text{expresión} \rangle \langle \text{expresión} \rangle)\end{aligned}$$

- Se puede representar cada expresión del cálculo lambda usando el tipo de dato `lc-exp` definido anteriormente mediante `define-datatype`



- Una BNF especifica una representación particular de un tipo de dato que usa los valores generados por la gramática
- Esta representación es llamada *sintaxis concreta* o *representación externa*
- Para procesar dichos datos, se requiere convertirlos a una *representación interna* o *sintaxis abstracta*, en la cual los símbolos terminales (como paréntesis) no necesitan ser almacenados ya que no llevan información



- Para crear una sintaxis abstracta a partir de una sintaxis concreta, se debe nombrar cada regla de producción de la sintaxis concreta y cada ocurrencia de un símbolo no terminal
- Para la gramática de las expresiones del cálculo λ , se puede resumir las opciones (sintaxis concreta y abstracta) usando la siguiente notación:

$\langle \text{expresión} \rangle ::= \langle \text{identificador} \rangle$
var-exp (id)

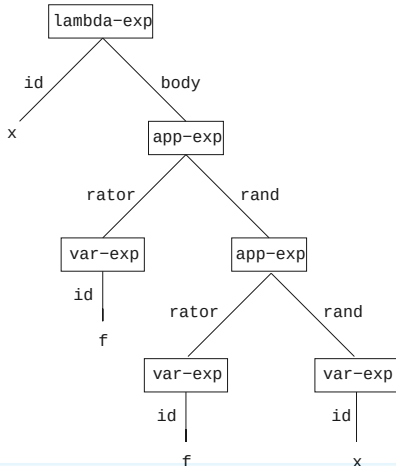
$::= (\text{lambda } (\langle \text{identificador} \rangle) \langle \text{expresión} \rangle)$
lambda-exp (id body)

$::= (\langle \text{expresión} \rangle \langle \text{expresión} \rangle)$
app-exp (rator rand)



- La sintaxis abstracta de una expresión es más fácil de comprender visualizándola como un *árbol de sintaxis abstracta*
- El siguiente ejemplo muestra el árbol para la expresión `(lambda (x) (f (f x)))`:





- Los árboles de sintaxis son útiles en lenguajes de programación de procesamiento de sistemas ya que los programas que procesan otros programas (como los interpretadores o compiladores) son casi siempre *dirigidos por sintaxis*
- Esto es que cada parte de un programa es guiado por la regla gramatical asociada con dicha parte, y cualquier subparte correspondiente a un símbolo no terminal puede ser accedido con facilidad



- Cuando se procesa la expresión $(\text{lambda } (x) (f (f x)))$, primero se debe reconocer como una expresión del cálculo lambda, correspondiente a la regla:

$$\langle \text{expresión} \rangle ::= (\text{lambda } (\langle \text{identificador} \rangle) \langle \text{expresión} \rangle)$$

- El parámetro formal es x y el cuerpo es $(f (f x))$. El cuerpo debe ser reconocido como una *app-exp*, y así sucesivamente.



El problema de convertir un árbol de sintaxis abstracta a una representación lista-y-símbolo (con lo cual Scheme mostraría las expresiones en su sintaxis concreta), se resuelve con el procedimiento:

```
(define unparse-expression
  (lambda (exp)
    (cases expression exp
      (var-exp (id) id)
      (lambda-exp (id body)
        (list 'lambda (list id)
              (unparse-expression body)))
      (app-exp (rator rand)
        (list (unparse-expression rator)
              (unparse-expression rand))))))
```



- La tarea de derivar el árbol de sintaxis abstracta a partir de una cadena de caracteres es denominado *parsing*, y es llevado a cabo por un programa llamado *parser* (analizador sintáctico)
- El siguiente procedimiento deriva la representación en sintaxis concreta a árboles de sintaxis abstracta:



```
(define parse-expression
  (lambda (datum)
    (cond
      ((symbol? datum) (var-exp datum))
      ((pair? datum)
       (if (eqv? (car datum) 'lambda)
           (lambda-exp (caadr datum)
                        (parse-expression (caddr datum)))
           (app-exp
            (parse-expression (car datum))
            (parse-expression (cadr datum)))))
      (else (eopl:error 'parse-expression
                          "Invalid concrete syntax ~s" datum)))))
```

